HWK 3

Applied Natural Language Processing

DATA 641

Converting Jupyter Notebooks to HTML

To convert your Jupyter Notebook to an HTML file, follow these steps:

1. Open the Jupyter Notebook that you want to convert to an HTML file.
2. Ensure that the notebook is saved with the latest changes.
3. In the Jupyter Notebook interface, navigate to **File** in the menu bar.
4. Click on **Download as** and select **HTML (.html)** from the drop-down menu. This will initiate the conversion process.
5. The Jupyter Notebook will be converted to an HTML file and automatically downloaded to the default download location on your computer.
6. Locate the downloaded HTML file on your computer and verify that the conversion was successful.
7. Optionally, open the HTML file in a web browser to view and verify the content.
8. Upload your HTML file on Canvas.

Make sure to save your Jupyter Notebook before attempting to convert it to an HTML file.

Exercise 1 (7 points) Given a collection of COVID-19-related posts from various social media platforms our task will be to pre-process, and generate various types of features that can be used to train a model and classify a post as either *real* or *fake*. Answer the following questions.

(a) (0.5 points) Access and import the data from the TrainLabels.csv file.

(b) (0.5 points) Preprocess the social media post dataset: Remove any special characters, URLs, and irrelevant symbols. Tokenize the posts into individual words or phrases. Convert the text to lowercase and remove stopwords if necessary.

(c) (1 points) One-hot encoding: Create a vocabulary of unique words across all the social media posts. Assign a binary value (1 or 0) to each word in the vocabulary, indicating whether it appears in a particular post. Encode each post using a one-hot encoding representation, where each word is represented by a binary feature. Store the one-hot encoded representations in a suitable data structure (e.g., a matrix or list).

(d) (1 points) Bag of words: Create a vocabulary of unique words across all the social media posts. Count the occurrences of each word in each post. Encode each post using a bag of words representation, where each word is represented by its frequency of occurrence. Store the bag of words representations in a suitable data structure (e.g., a matrix or list).

(e) (1 points) Bag of n-grams: Create a vocabulary of unique n-grams (phrases of n consecutive words) across all the social media posts. Count the occurrences of each n-gram in each post. Encode each post using a bag of n-grams representation, where each n-gram is represented by its frequency of occurrence. Store the bag of n-grams representations in a suitable data structure (e.g., a matrix or list).

(f) (1 points) TF-IDF (Term Frequency-Inverse Document Frequency): Calculate the term frequency (TF) of each word in each post, representing the frequency of a word within a specific post. Calculate the inverse document frequency (IDF) for each word, representing the importance of a word in the entire dataset. Compute the TF-IDF value for each word in each post, which combines the TF and IDF values. Encode each post using a TF-IDF representation, where each word is represented by its TF-IDF value. Store the TF-IDF representations in a suitable data structure (e.g., a matrix or list).

(g) (2 points) For the TF-IDF representations calculate pairwise cosine distances for each class: Identify the classes or labels present in the dataset (e.g., real,fake). For each class, calculate the pairwise cosine distances between the textual feature representations of the social media posts belonging to that class. Store the pairwise cosine distances for each class in a suitable data structure (e.g., a matrix or dictionary). From this matrix access the value that stores the highest similarity and identify those social media posts. Display those posts and discuss any insights or interesting findings from this comparison.

Exercise 2 (8 points) You have been given a dataset consisting of social media posts. Your task is to use a pre-trained Word2Vec model to generate word embeddings for each post and then visualize these embeddings using t-SNE and PCA techniques.

(a) (0.5 points) Load the pre-trained Word2Vec model: Download a pre-trained Word2Vec model (e.g., Google's Word2Vec model or any other suitable model). Then load the model into memory using a library of your choice (e.g., Gensim).

(b) (0.5 points) Preprocess the social media post dataset: Remove any special characters, URLs, and irrelevant symbols. Then tokenize the posts into individual words or phrases, and last, convert the text to lowercase and remove stopwords if necessary.

(c) (1 points) Generate word embeddings: For each post, retrieve the corresponding word vectors using the pre-trained Word2Vec model. Calculate the post embedding by averaging the word vectors. Store the post embeddings in a suitable data structure (e.g., a matrix or list).

(d) (1 points) Visualize the embeddings using t-SNE: Apply t-SNE (t-Distributed Stochastic Neighbor Embedding) to reduce the dimensionality of the post embeddings. Choose an appropriate number of dimensions for visualization. Plot the t-SNE visualizations of the post embeddings, using different colors or markers to represent different categories or labels if available.

(e) (1 points) Visualize the embeddings using PCA: Apply PCA (Principal Component Analysis) to further reduce the dimensionality of the post embeddings. Choose an appropriate number of principal components for visualization. Plot the PCA visualizations of the post embeddings, using different colors or markers to represent different categories or labels if available.

(f) (2 points) For the post embeddings calculate pairwise cosine distances for each class: Identify the classes or labels present in the dataset (e.g., real,fake). For each class, calculate the pairwise cosine distances between the post embeddings of the social media posts belonging to that class. Store the pairwise cosine distances for each class in a suitable data structure (e.g., a matrix or dictionary). From this matrix access the value that stores the highest similarity and identify those social media posts. Display those posts and discuss any insights or interesting findings from this comparison.

(g) (2 points) Analyze and interpret the visualizations: Examine the t-SNE and PCA visualizations to identify any clusters or patterns within the social media post embeddings. Discuss any insights or interesting findings from the visualizations. Reflect on the limitations of using t-SNE and PCA for visualizing high-dimensional word embeddings.

Note: You may need to install additional libraries, such as Gensim for Word2Vec, scikit-learn for t-SNE and PCA, and matplotlib for visualization.
Remember to document your code, provide explanations for each step, and include relevant plots or figures to support your findings.